IMPLEMENTATION ARCHITECTURE | READING GUIDE

STATUS

Proposed target architecture for joint review, not authorization to implement. The user-approved direction is React PWA -> FastAPI -> Supabase PostgreSQL with separately hosted frontend and backend.

DEPLOYMENT

One FastAPI deployment per client initially. Master Core distributes updates by reviewed upstream Git merges into isolated client source sets and environments.

MODULARITY

FastAPI is a modular monolith: identity/RBAC, employees/hierarchy, attendance, field DCR, visits and proofs, leave/payroll, tasks, reports, notifications, Socket.IO.

SECURITY

FastAPI must authenticate, enforce DB-defined role permissions and employee/territory resource scope, validate inputs, and own protected business mutations. Supabase RLS is not the primary application authorization method.

PERSISTENCE

Supabase PostgreSQL hosts hashed passwords and domain records. Existing photos are reported as Base64 in TEXT fields with pending browser localStorage caching; private Supabase Storage was discussed but is not the established existing implementation.

SAFETY

Live pilot data exists. Validate backups, active access grants, duplicate/orphan records, contracts, and recovery before migration. Do not shut down legacy browser PostgREST access before safe functional cutover.

DOMAIN OWNERSHIP AND IMPLEMENTATION SEQUENCE

FOUNDATION

Authentication, login fallback verification, session life cycle, RBAC permission/resource scope, common API contracts, audit events, error reporting.

EMPLOYEE AND HIERARCHY

Employee directory, reporting tree, territory authority, payroll field masking and safe authorization.

CORE FIELD OPERATIONS

Attendance sessions, geofence evaluation, DCR, doctor/chemist visits, missed-punch corrections and photo evidence. Model concurrency and retries before writes.

HR OPERATIONS

Leave balance consistency, approval rules, advances, attendance finalization and payroll calculation with transactions and audit trails.

SUPPORTING SERVICES

Tasks, reporting, notifications, Socket.IO chat, background work only when justified. Durable records and missed-event reconciliation are needed.

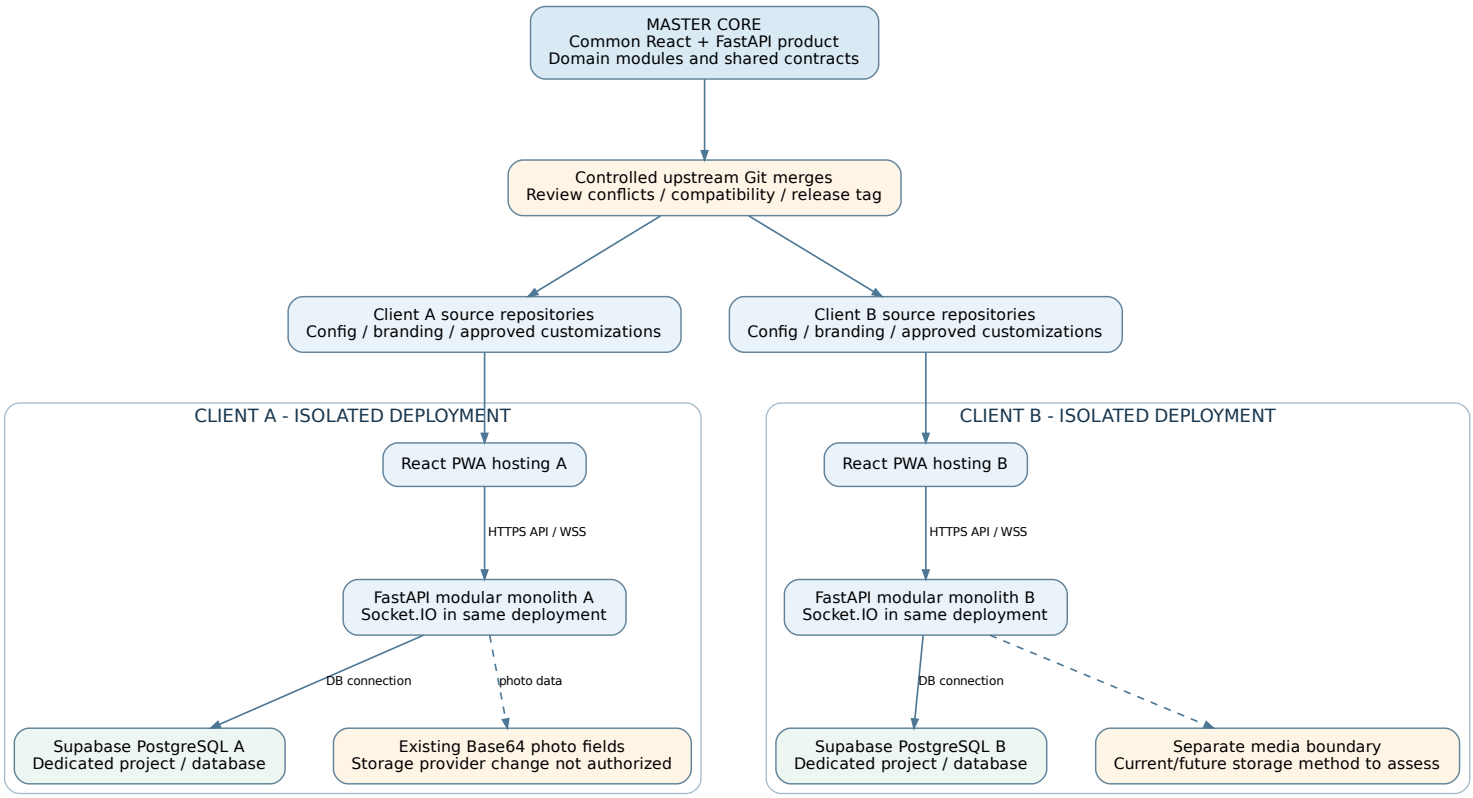
CLIENT READINESS

Core version tagging, controlled upstream Git merging, extensions vs config classification, database version compatibility, onboarding, ownership and recovery plans.

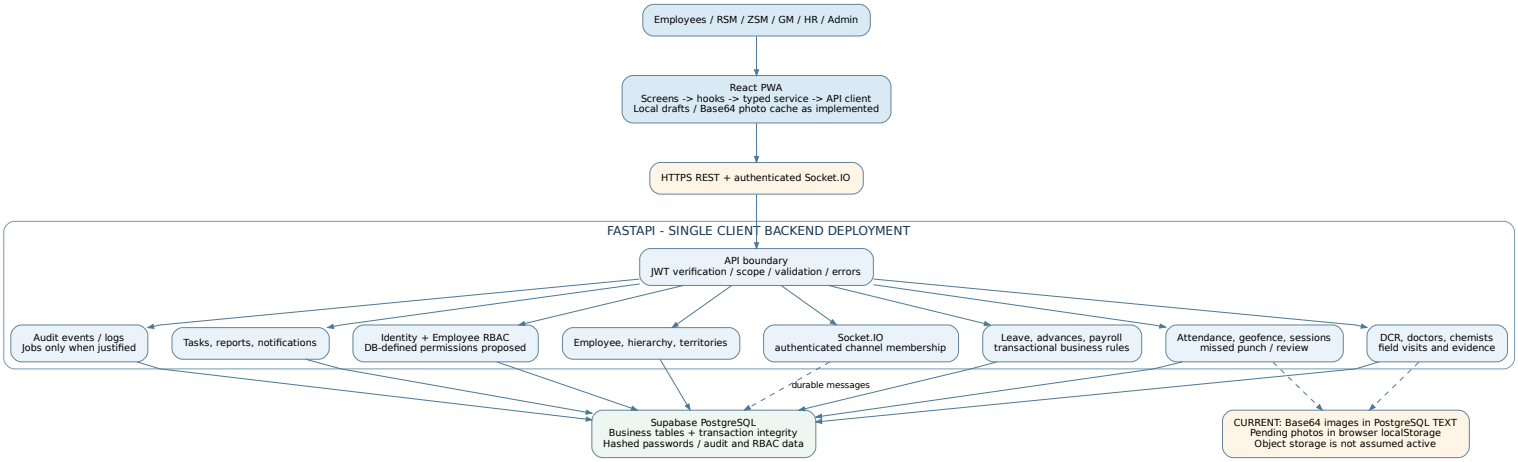
OPEN DECISIONS KEPT OUTSIDE DIAGRAMS

Final photo storage strategy, offline retry persistence, client handover/ownership, feature entitlement switching, exact RBAC matrix, live Supabase privileges, backup policy and A7 edge cases remain subjects for review.

1 | MASTER CORE AND CLIENT ISOLATION



2 | PER-CLIENT APPLICATION RUNTIME



3 | AUTHORIZATION AND BUSINESS TRANSACTION PATHS

